MailCheck – Bulk Email Verification SaaS

Software Development Assignment Submission Document

Project Title: MailCheck – Bulk Email Verification Tool | **Framework:** React 19 + Flask 3.1 + SQLAlchemy
Runtime Environment: Linux (Render) / Windows | **Python:** 3.11.10
Classification Standard: RFC 5321 / RFC 5322 / RFC 7505 | **Status:** Fully Implemented, Tested & Deployed

---

1. Project Links & Publication Status

- **Live Website URL:** https://mailcheck-email-verifier.onrender.com
- **GitHub Repository URL:** https://github.com/VenkatRoyal30/mailcheck-email-verifier
- **Publication Status:** Source code published on GitHub and application deployed on Render.

2. Problem Statement & Core Objectives

High email bounce rates degrade domain reputation and cause sender blacklisting by major mailbox providers. MailCheck is an evidence-based bulk email verification platform that verifies email syntax compliance, checks authoritative DNS mail routing (MX and RFC 5321 fallback A/AAAA records), and reports one of three definitive outcomes:

1. **Likely to be delivered:** Valid syntax and confirmed active mail exchange (MX) or fallback routing.
2. **Likely to bounce:** Invalid syntax, nonexistent domain (NXDOMAIN), RFC 7505 Null MX, or confirmed absence of mail routes.
3. **Unknown:** Inconclusive evidence due to DNS query timeout, SERVFAIL, temporary resolver failure, or greylisting.

Important Disclosure: These outcomes are evidence-based classifications, not delivery guarantees.

3. Technology Stack

- **Frontend:** React 19, TypeScript, Vite 6, Tailwind CSS (light/dark theme), Lucide Icons, Typed REST API Client.
- **Backend:** Python 3.11.10, Flask 3.1 REST API, Gunicorn 23.0 WSGI Server, email-validator (RFC 5322/6531), dnspython (MX/Null MX/Fallback), pytest.
- **Database:** SQLAlchemy 2.0 ORM with SQLite for local development and PostgreSQL support through DATABASE_URL.

4. Verification Workflow Pipeline

Stage 1: RFC Format & Syntax: Validates input against RFC length limits (local part ≤ 64 octets, address ≤ 254 octets), detects missing/multiple '@' separators, and checks illegal whitespace. Invalid syntax is immediately categorized as 'Likely to bounce', skipping DNS and SMTP.
Stage 2: DNS & MX Mail Routing: Queries MX records with strict timeouts. Detects RFC 7505 Null MX records. If MX records are absent, applies RFC 5321 fallback to A/AAAA records. Resolves NXDOMAIN as 'Likely to bounce' and resolver timeouts/SERVFAIL as 'Unknown'.
Stage 3: Controlled SMTP Probing: Disabled by default to respect port 25 constraints. When enabled, issues non-intrusive RCPT TO checks without sending email bodies. Distinguishes definitive non-existence (5.1.1) from ambiguous policy/relay rejections (which remain Unknown). Detects catch-all servers.

5. Persistent Database Storage & Privacy Policy

- **Entities:** VerificationBatch (UUID, creation timestamp, total/delivered/bounce/unknown counts, SMTP mode) and EmailVerification (UUID, batch ID, raw email, normalized email, status, diagnostics).
- **Data Retention & Deletion:** 30-day configurable lifecycle. Provides full user controls to delete individual records (with batch count recalculation) or purge entire batches.
- **Privacy Disclosure:** Single-user private demo deployment. No email messages are ever sent, and no credentials or passwords are collected.

6. Automated Testing Summary (Actual Test Execution)

All 58 backend and integration tests executed and passed via pytest with mocked DNS and socket layers:

Test Module	Scope & Test Focus	Tests	Outcome
test_syntax.py	RFC syntax validation, separators, length constraints, illegal whitespace, malformed labels	10	10 PASSED
test_dns.py	MX records, RFC 7505 Null MX, RFC 5321 A/AAAA fallback, NXDOMAIN, DNS timeouts, SERVFAIL	7	7 PASSED
test_smtp.py	Probe, catch-all detection, 550 rejection, 451 greylisting, port 25 timeout, Gmail timeout, policy/relay Unknown, definitive 5.1.1	16	16 PASSED
test_pipeline.py	Pipeline stage skipping, default disabled SMTP, batch duplicates, limit, working-format Gmail timeout, regression suite	11	11 PASSED
test_api_verify.py	POST /api/verify, input validation, multiline input, batch limits, database persistence	6	6 PASSED
test_api_history.py	GET /api/history, batch detail, single record deletion with count recalculation, CSV export	1	1 PASSED
test_frontend_routes.py	GET /, SPA client routes, asset delivery, unknown /api 404 JSON, root WSGI app export, PostgreSQL URL normalization	7	7 PASSED
TOTALS	**Comprehensive test suite with 100% pass rate in 3.24s**	**58**	**58 PASSED**

Frontend Production Build: npm run build succeeded with 0 TypeScript errors. Output generated in frontend/dist/.